System Design & Architecture

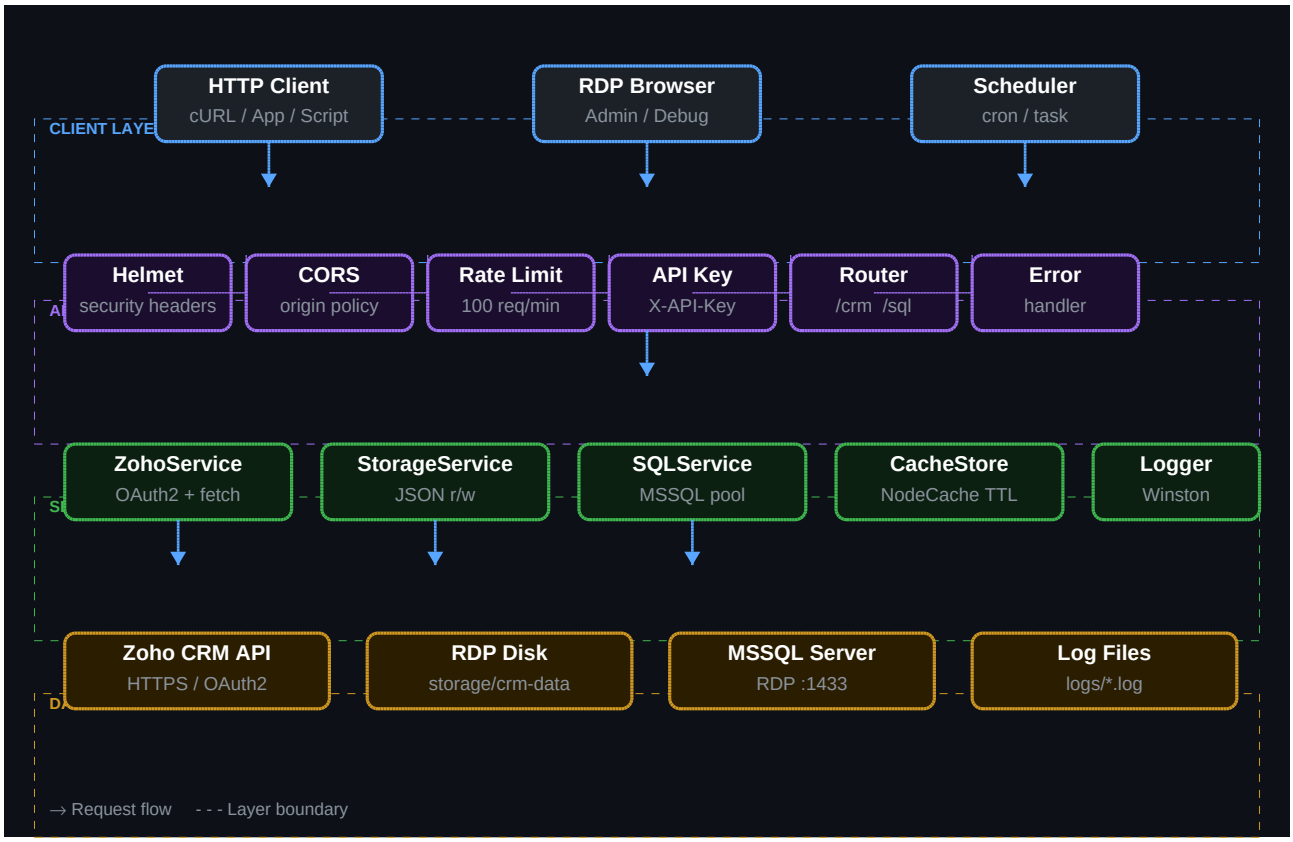
Production-grade CRM Data Pipeline on RDP Server

Version 1.0 • March 09, 2026

Stack	`Node.js 18+ / Express 4`
CRM Source	`Zoho CRM v2 API (OAuth2)`
Database	`Microsoft SQL Server (MSSQL)`
Storage	`Local JSON files on RDP disk`
Logging	`Winston + Daily Rotate (3 streams)`
Auth	`API Key (X-API-Key header)`
Rate Limit	`100 req/min | 10 req/min (sync)`
Design Pattern	`SOLID • Singleton Services • Factory App`

1. High-Level Architecture

sandbox-api is a layered REST API server deployed on the RDP server. Clients send authenticated HTTP requests which pass through a security gateway layer before reaching controller → service → data store.



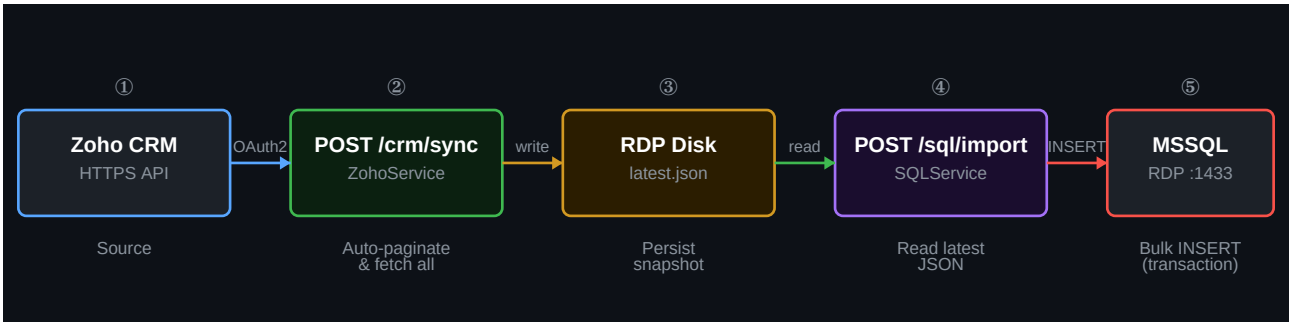
The architecture is intentionally stateless at the HTTP layer — all state lives in the CacheStore (in-memory), the RDP disk (JSON files), or the MSSQL database.

2. Layer Breakdown

Layer	Files	Responsibility
Entry Point	server.js	Bootstrap app, bind port, handle SIGTERM/SIGINT gracefully
App Factory	src/app.js	Wire middleware pipeline: Helmet → CORS → body-parse → request ID → Morg
Config	src/config/env.js	Centralised env access. All process.env reads happen here — nowhere else
Routes	src/routes/*.js	Thin route definitions. Attach middleware (rate limiters) and delegate to controller
Controllers	src/controllers/*.js	Validate input (Joi), call service methods, shape HTTP response. No business lo
Services	src/services/*.js	All business logic and I/O. ZohoService, SQLService, StorageService are singlet
Store	src/store/CacheStore.js	In-memory TTL cache for Zoho tokens and hot data. Singleton wrapping NodeCa
Middleware	src/middleware/*.js	Cross-cutting concerns: rate limiting, API key auth, global error handler
Utils	src/utils/*.js	Logger (Winston + rotation) and Validator (Joi schemas) — no side effects

3. CRM → SQL Data Pipeline

The primary use case is a two-step pipeline: first sync CRM data to disk, then import from disk into MSSQL. This decouples the Zoho dependency from the SQL write, making each step independently retryable.



① **Zoho CRM** — Source of truth. Accessed via HTTPS with a cached OAuth2 access token (auto-refreshed on expiry or 401).

② **POST /crm/sync** — ZohoService auto-paginates until more_records=false, collecting all records into memory.

③ **RDP Disk** — StorageService writes latest.json (overwritten) + a timestamped snapshot (immutable). Snapshots pruned after 30 days.

④ **POST /sql/import** — SQLService reads latest.json and begins a SQL Server transaction.

⑤ **MSSQL** — All rows are inserted atomically. If any row fails, the entire batch is rolled back via transaction.rollback().

4. RDP Storage Layout

All CRM data is persisted as structured JSON under storage/crm-data/. Each module gets its own directory. Snapshots provide a full audit trail.

```
storage/crm-data/                          ← auto-created on first sync
    Leads/
        latest.json
        snapshots/                         ← overwritten each sync
            2025-03-09T10-30-00.json
            2025-03-08T09-15-00.json       ← never overwritten, pruned >30d
            ...
    Contacts/
        latest.json
        snapshots/
            ...
    Accounts/
        ...
```

JSON Envelope Format

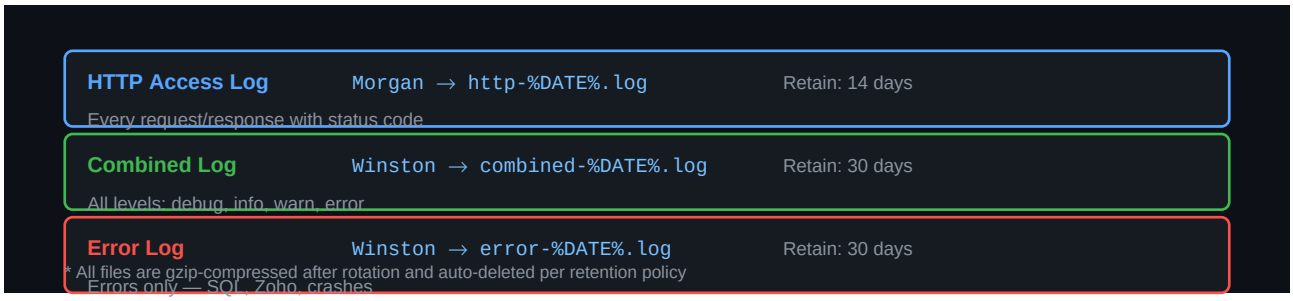
```
{
"module": "Leads",
"fetchedAt": "2025-03-09T10:30:00.000Z",
"count": 1842,
"records": [ { "id": "...", "Last_Name": "Smith", ... }, ... ]
}
```

5. Security Model

Control	Implementation	Notes
API Key Auth	rateLimiter.js	X-API-Key header validated on every /api/v1/* request
Security Headers	helmet.js	X-Frame-Options, CSP, HSTS, X-Content-Type-Options
CORS	cors.js	Origin whitelist in production (open in dev)
Rate Limiting	express-rate-limit	100 req/min standard; 10 req/min for sync/import endpoints
SQL Injection	mssql parameterised	All SQL values passed as typed parameters — no string concat
TLS	.env SQL_ENCRYPT	SQL Server connection encrypted; trust cert configurable
Token Storage	CacheStore (memory)	Zoho tokens never written to disk. Expire from cache naturally
Body Size Limit	express.json	10 MB max body to prevent memory exhaustion attacks
Error Exposure	errorHandler.js	5xx errors only show generic message in production

6. Logging Architecture

Three independent daily-rotating log streams ensure observability without mixing concerns. All files are JSON-structured for easy parsing by log aggregators (Datadog, ELK, etc.).



Log Level Hierarchy

error → warn → info → http → debug. In production, LOG_LEVEL=info suppresses debug output. Console output is disabled in NODE_ENV=production.

Structured Log Fields

Field	Type	Present in
`timestamp`	string	All logs
`level`	string	All logs
`message`	string	All logs
`requestId`	string	Error logs (from req.id uuid)
`method`	string	Error logs
`url`	string	Error logs
`statusCode`	integer	Error logs
`stack`	string	Error logs in dev only

7. SOLID Principles Applied

S	**Single Responsibility** server.js bootstraps only. app.js wires only. ZohoService fetches only. StorageService stores only. Each class/module has exactly one reason to change.
O	**Open / Closed** SQLService.select(), bulkInsert(), query() can all be called by any consumer. New query types are added as new methods — existing ones are never modified.
L	**Liskov Substitution** All services are singletons that can be replaced with mock implementations in tests without changing controller logic. jest.mock('../services/ZohoService') just works.
I	**Interface Segregation** CRMController only imports ZohoService and StorageService. SQLController only imports SQLService and StorageService. No controller imports everything.
D	**Dependency Inversion** app.js depends on route modules, not concrete controllers. Controllers depend on service abstractions. env.js is the single source — nothing reads process.env directly.

8. Error Handling Strategy

Every error passes through a single central errorHandler middleware. Services throw errors freely; controllers catch and forward with next(err). AppError carries statusCode, code, and an expose flag for safe message surfacing.

```
// In a controller
try {
const data = await ZohoService.fetchRecords(opts);
res.json({ success: true, data });
} catch (err) {
next(err); // ← single line, always
}

// AppError factory
throw new AppError('Record not found', 404, 'NOT_FOUND');
// → expose=true (4xx): message shown to client
// → expose=false (5xx): generic message shown, real message in logs
```

Graceful Shutdown

server.js listens for SIGTERM and SIGINT. On signal, server.close() stops accepting new connections and allows in-flight requests to complete. A 10-second forced exit timer prevents hanging processes during deployment or restart.

Zoho Token Recovery

If Zoho returns a 401 Unauthorized response, ZohoService deletes the cached token and throws the error. The next call will automatically trigger a token refresh — no manual intervention required.

9. Deployment on RDP Server

Step	Command / Action
1. Clone / copy project	Copy sandbox-api/ folder to RDP server
2. Install dependencies	npm install --production
3. Configure environment	Edit .env with SQL credentials, Zoho tokens, API key
4. Create log directory	mkdir -p logs (auto-created by Winston on first run)
5. Start server	node server.js OR npm start
6. Run as service (Windows)	Use NSSM or PM2 to run as a background Windows service
7. Run as service (Linux)	Create a systemd unit or use PM2: pm2 start server.js
8. Verify health	curl http://localhost:3000/health
9. Test sync	POST /api/v1/crm/sync { "module": "Leads" }

Recommended PM2 Config (ecosystem.config.js)

```
module.exports = {
apps: [{
name : 'sandbox-api',
script : 'server.js',
instances : 1,
autorestart: true,
watch : false,
max_memory_restart: '500M',
env_production: {
NODE_ENV: 'production',
PORT: 3000
}
}]
};
```